Assignment No. 1

Title :- Python installation and configuration with windows and Linux

Date:-

Marks:- Signature:-

Assignment No. 1

Problem Statement: - Python installation and configuration with windows and Linux

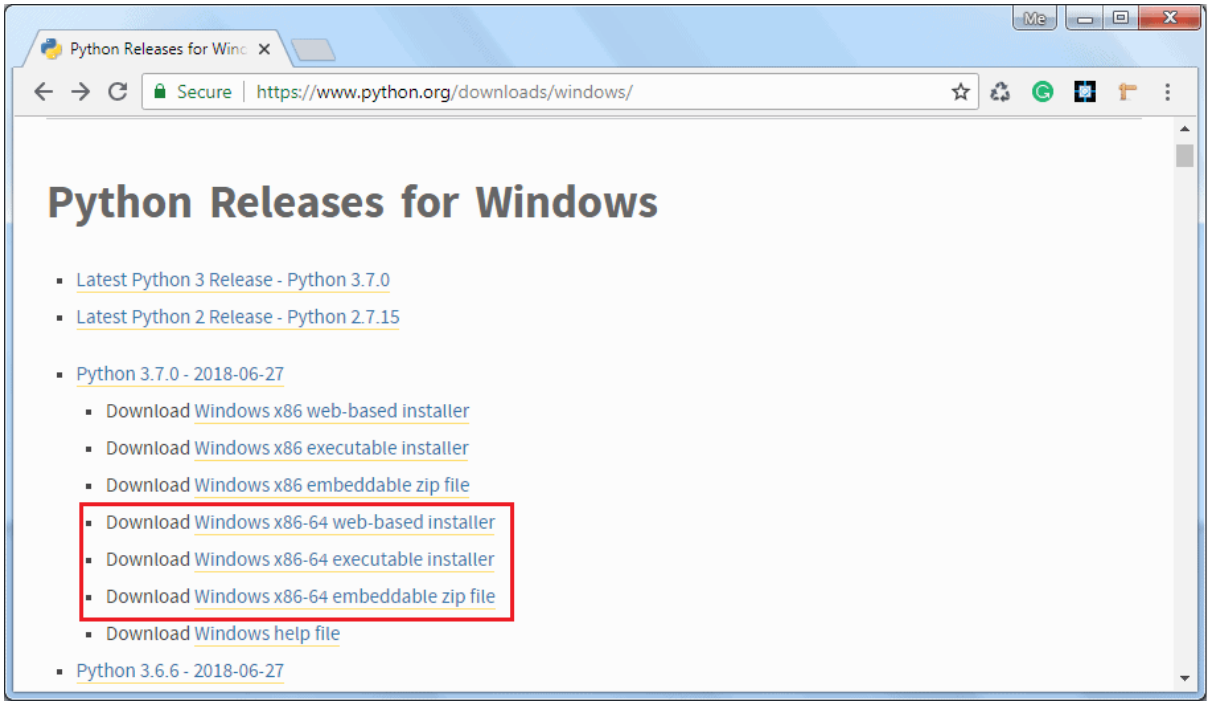
Objective: - To Learn Python installation and configuration with windows and Linux Theory

Introduction: Python can be installed on Windows, Linux, Mac OS as well as certain other platforms such as IBM AS/400, iOS, Solaris, etc.

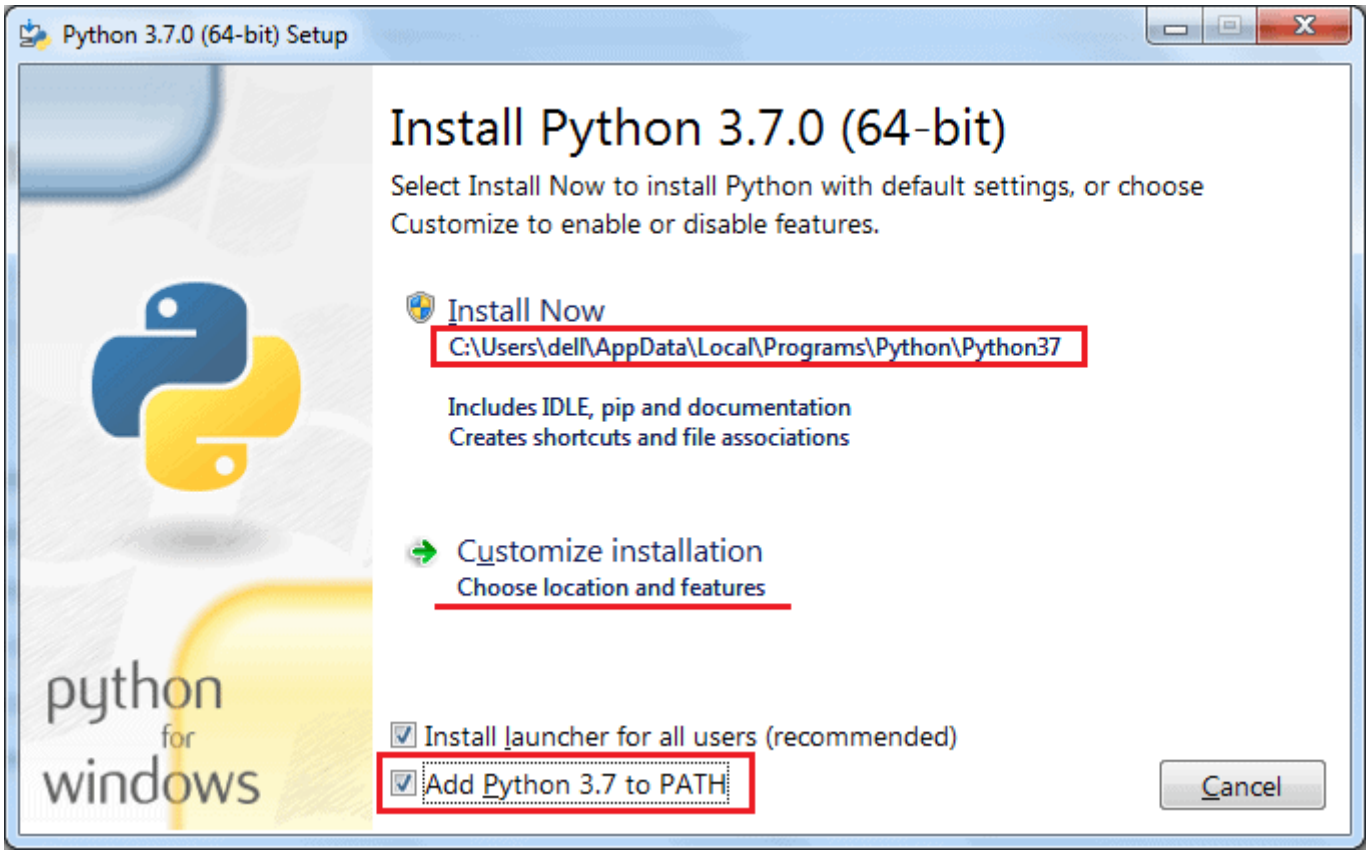
To install Python on your local machine, get a copy of the standard distribution of Python software from https://www.python.org/downloads based on your operating system, hardware architecture and version of your local machine.

Install Python on Windows: To install Python on a Windows platform, you need to download the installer. A web-based installer, executable installer and embeddable zip files are available to install Python on Windows. Visit https://www.python.org/downloads/windows and download the installer based on your local machine's hardware architecture.

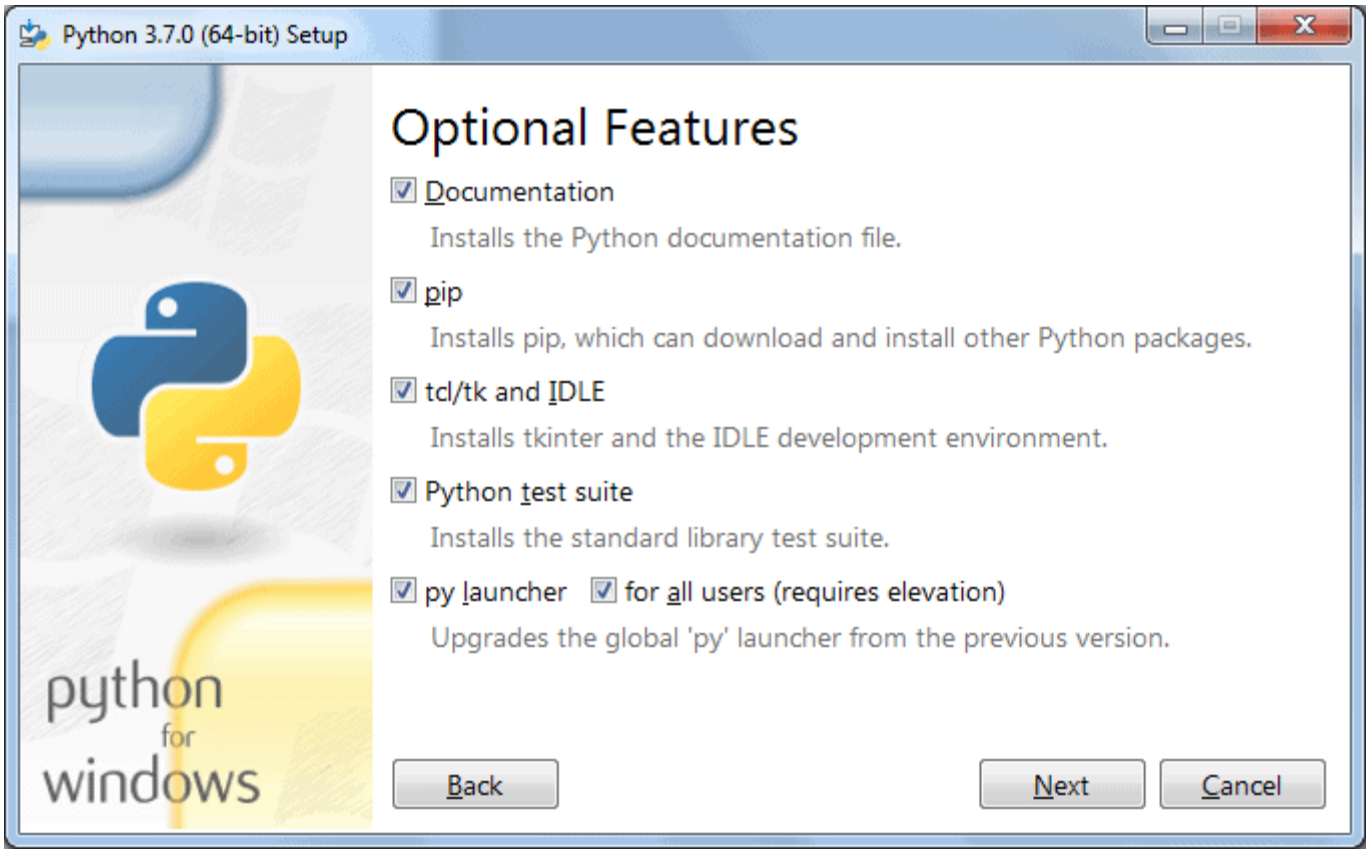
The web-based installer needs an active internet connection. So, you can also download the standalone executable installer. Visit https://www.python.org/downloads and click on the Download Python 3.7.0 button as shown below. (3.7.0 is the latest version as of this writing.)



> Download the Windows x86-64 executable installer and double click on it to start the python installation wizard as shown below.

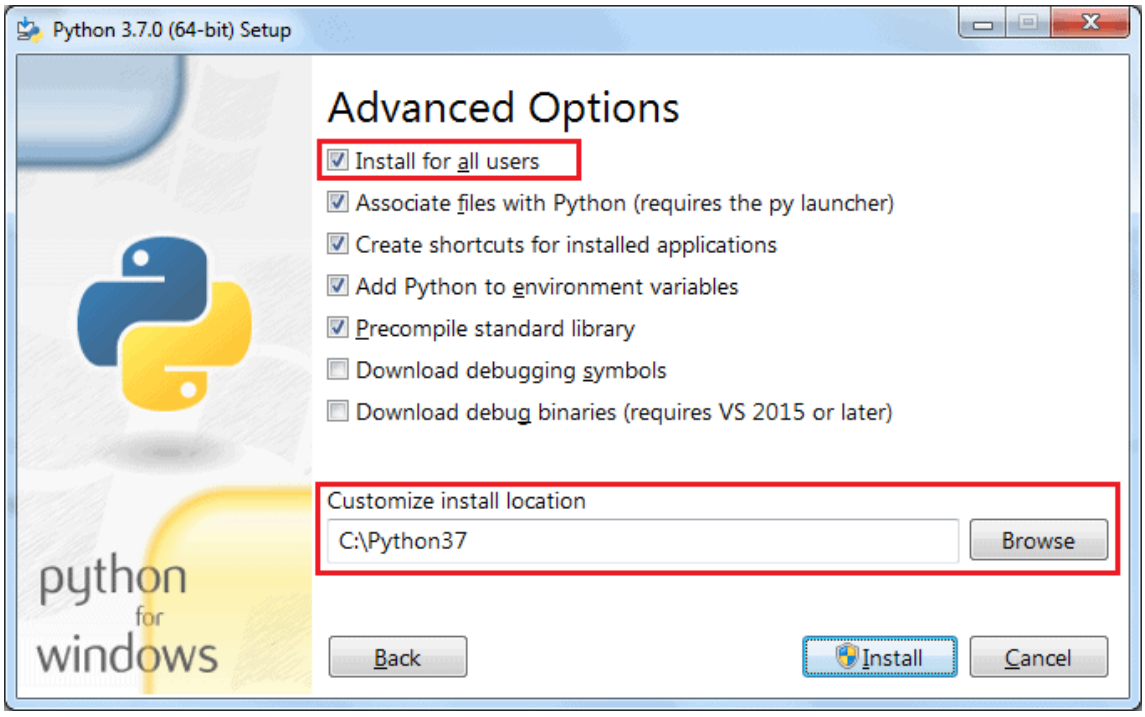


Installation is a simple wizard-based process. As you can see in the above figure, the default installation folder will be C:\ Users\ {UserName}\ AppData\ Local\Programs\ Python\ Python37 for Python 3.7.0 64 bit. Check the Add Python 3.7 to PATH checkbox, so that you can execute python scripts from any path. You may choose the installation folder or feature by clicking on Customize installation. This will go to the next step of optional features, as shown below.



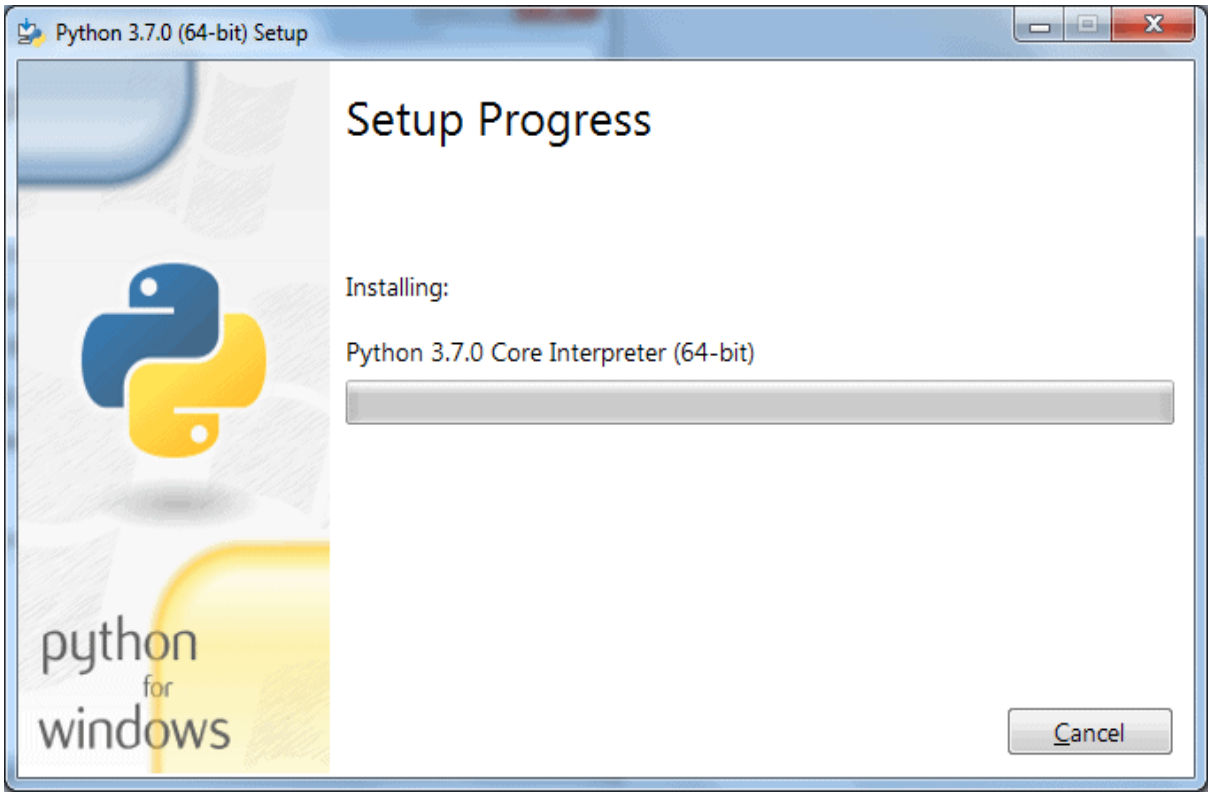
Python Installation Wizard

Click Next to continue.



Python Installation Wizard

In Advanced Options, select the Install for all users option so that any user of your local machine can execute Python scripts. Also, choose the installation folder to make a shorter path for Python executable (something like C:\python37), keeping the rest of the choices to default and finally click on the Install button.



Python Installation Wizard

After successful installation, you can check the Python installation by opening a command prompt and type python --version or python -V and press Enter. If Python installed successfully then it will display the installed version.

Install Python on Linux

Most of Linux distributions come with Python already installed. However, the Python 2.x version is incorporated in many of them. To check if Python 3.x is available, run the following command in the Linux terminal:

$ which python3

If available, it will return the path to the Python3 executable as /usr/local/bin/python3.

To install Python on Ubuntu 18.04, Ubuntu 20.04 and above, execute the following

commands:

$ sudo apt-get update
$ sudo apt-get install python3.7 python3-pip

After the installation, you can run Python 3.8 and pip3 commands.

For other Linux distributions use the corresponding package managers, such as YUM for Red Hat, aptitude for debian, DNF for Fedora, etc.

For installation on other platforms as well as installation from the source code,

QUESTION:-

1. What is the difference between list and tuples in Python?
2. What are the key features of Python??
3. How is Python an interpreted language?
4. How is Python an interpreted language?
5. What are the basic steps of Python?

CONCLUSION:- ----------

Assignment No. 2

Title :- Write Programs for understanding the data types, control flow statements, blocks and loops

Date:-

Marks:- Signature:-

Assignment No. 2

Problem Statement: - Learn Programs for understanding the data types, control flow statements, blocks and loops

Objective: - Learn Programs for understanding the data types, control flow statements, blocks and loops

Theory :

Introduction:

Python Data Types :There are different types of data types in Python. Some built-in Python data types are:

1. Numeric data types: int, float, complex
2. String data types: str
3. Sequence types: list, tuple, range
4. Binary types: bytes, bytearray, memoryview
5. Mapping data type: dict
6. Boolean type: bool
7. Set data types: set, frozenset

Python Numeric Data Type : Python numeric data type is used to hold numeric values like;

1. int – holds signed integers of non-limited length.
2. long- holds long integers(exists in Python 2.x, deprecated in Python 3.x).
3. float- holds floating precision numbers and it's accurate up to 15 decimal places.
4. complex- holds complex numbers.

Python Numeric Data Type :

```
#create a variable with integer value.

a=100 print("The type of variable having value", a, " is ", type(a))

#create a variable with float value. b=10.2345 print("The type of
variable having value", b, " is ", type(b))

#create a variable with complex value. c=100+3j print("The type of
variable having value", c, " is ", type(c))
```

Python String Data Type

The string is a sequence of characters. Python supports Unicode characters. Generally, strings are represented by either single or double-quotes.

Python List Data Type

The list is a versatile data type exclusive in Python. In a sense, it is the same as the array in C/C++. But the interesting thing about the list in Python is it can simultaneously hold different types of data. Formally list is an ordered sequence of some data written using square brackets([]) and commas(,)

```
#list of having only integers a= [1,2,3,4,5,6] print(a) #list of having only strings b=["hello","john","reese"] print(b) #list of having both integers and strings c= ["hey","you",1,2,3,"go"] print(c)

#index are 0 based. this will print a single character print(c[1]) #this will print "you" in list c
```

Python Tuple

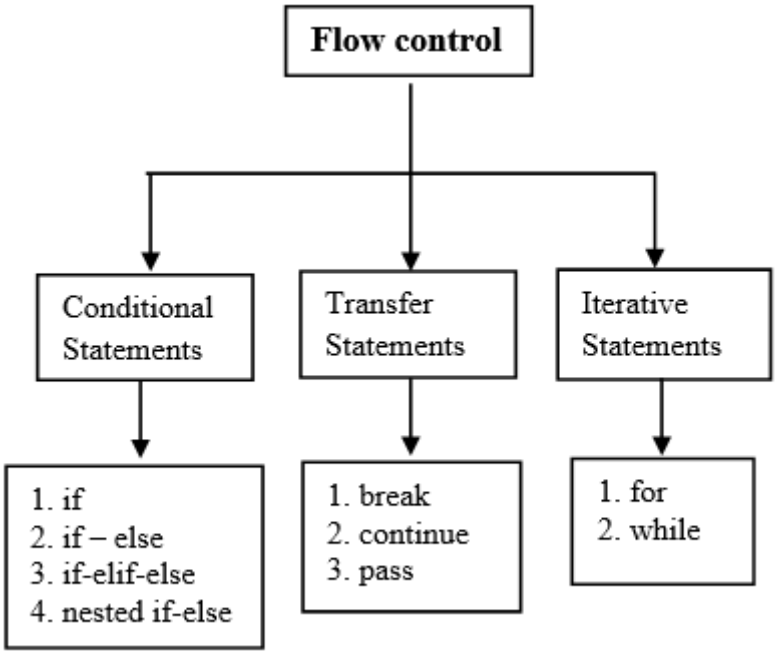
The tuple is another data type which is a sequence of data similar to a list. But it is immutable. That means data in a tuple is write-protected. Data in a tuple is written using parenthesis and commas

```
#tuple having only integer type of data. a=(1,2,3,4)

print(a) #prints the whole tuple #tuple having multiple type of data. b=("hello", 1,2,3,"go")

 print(b) #prints the whole tuple #index of tuples are also 0 based. print(b[4]) #this prints a single element in a tuple, in this case "go"
```

Python Control Flow Statements and Loops : In Python programming, flow control is the order in which statements or blocks of code are executed at runtime based on a condition.

The flow control statements are divided into three categories

1. Conditional statements
2. Iterative statements.
3. Transfer statements



Conditional statements

In Python, condition statements act depending on whether a given condition is true or false. You can execute different blocks of codes depending on the outcome of a condition. Condition statements always evaluate to either True or False.

There are three types of conditional statements.

1. if statement
2. if-else
3. if-elif-else
4. nested if-else

Iterative statements

In Python, iterative statements allow us to execute a block of code repeatedly as long as the condition is True. We also call it a loop statements.

Python provides us the following two loop statement to perform some actions repeatedly

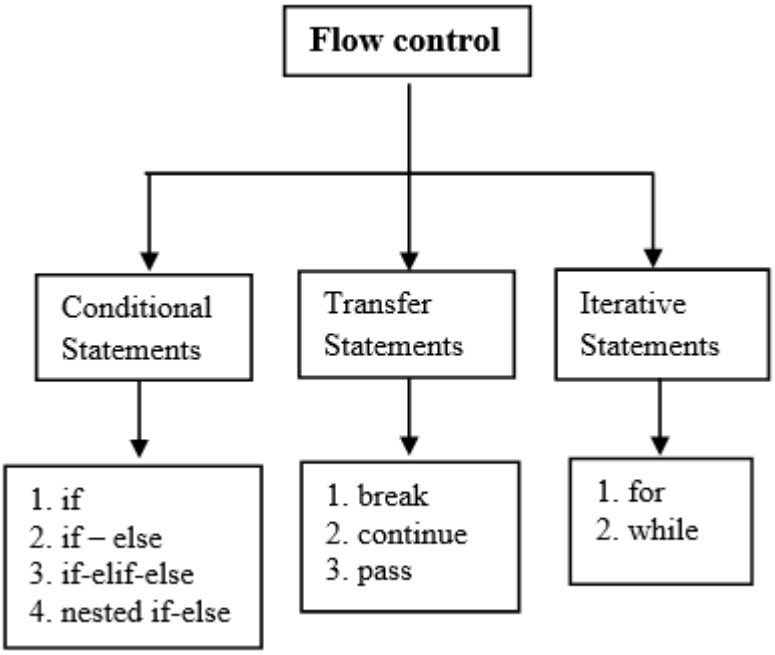
1. for loop
2. while loop
3. Let's see the example of the if statement. In this example, we will calculate the square of a number if it greater than 5
4. **Example**

```
number = 6
if number > 5:
    # Calculate square
    print(number * number)
print('Next lines of code')
```

Python Control Flow Statements and Loops :In Python programming, flow control is the order in which statements or blocks of code are executed at runtime based on a condition.

Control Flow Statements :The flow control statements are divided into three categories

1. Conditional statements
2. Iterative statements.
3. Transfer statements



Python control flow statements

Conditional statements

In Python, condition statements act depending on whether a given condition is true or false. You can execute different blocks of codes depending on the outcome of a condition. Condition statements always evaluate to either True or False.

There are three types of conditional statements.

1. if statement
2. if-else
3. if-elif-else
4. nested if-else

Iterative statements

In Python, iterative statements allow us to execute a block of code repeatedly as long as the condition is True. We also call it a loop statements.

Python provides us the following two loop statement to perform some actions repeatedly

for loop

while loop

Transfer statements

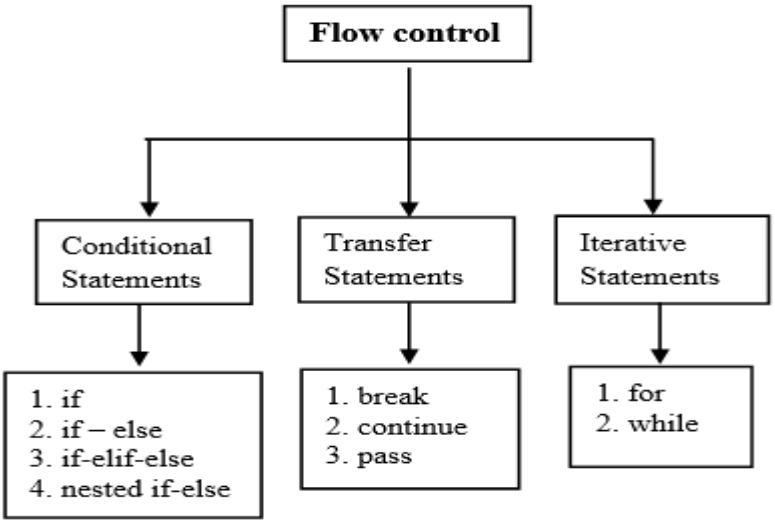
In Python, transfer statements are used to alter the program's way of execution in a certain manner. For this purpose, we use three types of transfer statements.

1. break statement
2. continue statement
3. pass statements

If statement in Python :**Control Flow Statements**

The flow control statements are divided into three categories

1. Conditional statements
2. Iterative statements.
3. Transfer statements



Python control flow statements

Conditional statements: In Python, condition statements act depending on whether a given condition is true or false. You can execute different blocks of codes depending on the outcome of a condition. Condition statements always evaluate to either True or False.

There are three types of conditional statements.

1. if statement

2. if-else
3. if-elif-else
4. nested if-else

Iterative statements :In Python, iterative statements allow us to execute a block of code repeatedly as long as the condition is True. We also call it loop statements.

Python provides us the following two loop statement to perform some actions repeatedly

1. for loop
2. while loop

In Python, transfer statements are used to alter the program's way of execution in a certain manner.

1. break statement
2. continue statement
3. pass statements

If statement in Python : In control statements, The if statement is the simplest form. It takes a condition and evaluates to either True or False.

If the condition is True, then the True block of code will be executed, and if the condition is False, then the block of code is skipped, and The controller moves to the next line

Syntax of the if statement

```
if condition:
     statement 1
     statement 2
     statement n
```

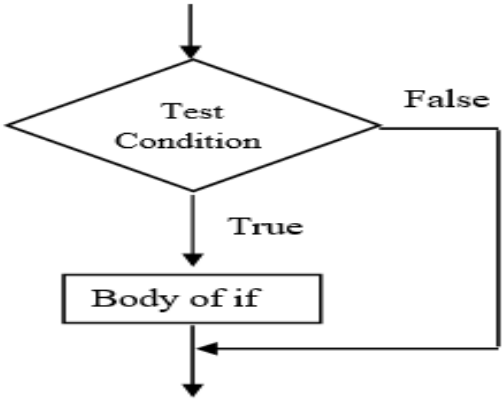


Fig. Flowchart of if statement

Example

```
number = 6
if number > 5:
    # Calculate square
    print(number * number)
print('Next lines of code')
```

QUESTION :-

1. What are the different data types in Python?
2. What is set data type in Python?
3. What is a character data type in Python?
4. What are the control statements in Python?
5. What is flow control in Python?

CONCLUSION:- ----------

Assignment No. 3

Title :- Programs for understanding functions, use of built in functions, user defined functions

Date:-

Marks:- Signature:-

Assignment No. 3

Problem Statement: - Write Program for understanding functions, use of built in functions, user defined functions

Objective: - To Learn Programs for understanding functions, use of built in functions, user defined functions

Theory :

Introduction A function is a set of statements that take inputs, do some specific computation and produce output. The idea is to put some commonly or repeatedly done tasks together and make a function so that instead of writing the same code again and again for different inputs, we can call the function.
Functions that readily come with Python are called built-in functions. Python provides built-in functions like print (), etc. but we can also create your own functions.

These functions are known as **user defines functions**.

User defined functions

All the functions that are written by any us comes under the category of user defined functions. Below are the steps for writing user defined functions in Python.

- In Python, def keyword is used to declare user defined functions.
- An indented block of statements follows the function name and arguments which contains the body of the function.

Syntax:

```
def function_name():
    statements
```

Parameterized Function
The function may take arguments(s) also called parameters as input within the opening and closing parentheses, just after the function name followed by a colon.

Syntax:

def function_name(argument1, argument2, ...):

statements

.

Example:

```
# Python program to

# demonstrate functions

# A simple Python function to check

# whether x is even or odd

def evenOdd( x ):

    if (x % 2 == 0):

        print("even")

    else:

        print("odd")

# Driver code

evenOdd(2)

evenOdd(3)
```

Default arguments :A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument.

Keyword arguments :The idea is to allow caller to specify argument name with values so that caller does not need to remember order of parameters.

Python User-defined Functions : Functions that we define ourselves to do certain specific task are referred as user-defined functions. The way in which we define and call functions in Python are already discussed. Functions that readily come with Python are called built-in functions. If we use functions written by others in the form of library, it can be termed as library functions. All the other functions that we write on our own fall under user-defined functions. So, our user-defined function could be a library function to someone else.

Advantages of user-defined functions

1. User-defined functions help to decompose a large program into small segments which makes program easy to understand, maintain and debug.
2. If repeated code occurs in a program. Function can be used to include those codes and execute when needed by calling that function.
3. Programmers working on large project can divide the workload by making different functions.

Example of a user-defined function

```
# Program to illustrate
# the use of user-defined functions

def add_numbers(x,y):
    sum = x + y
    return sum
```

```
num1 = 5
num2 = 6

print("The sum is", add_numbers(num1, num2))
```

Question :-

1. What do you mean by built-in functions and user-defined functions?
2. What is user-defined function with example?
3. How do you define a user defined function in Python?
4. Where is function defined in Python?
5. What is the difference between user defined and built-in functions in Python?

CONCLUSION:- ----------

Assignment No. 4

Title :- Learn Programs to use existing modules, packages and creating modules, packages

Date:-

Marks:- Signature:-

Assignment No. 4

Problem Statement: - Learn Programs to use existing modules, packages and creating modules, packages

Objective: - Understand Programs to use existing modules, packages and creating modules, packages

Theory:

Python modules and Python packages, two mechanisms that facilitate modular programming.

Modular programming refers to the process of breaking a large, unwieldy programming task into separate, smaller, more manageable subtasks or modules. Individual modules can then be cobbled together like building blocks to create a larger application.

There are several advantages to modularizing code in a large application:

Simplicity: Rather than focusing on the entire problem at hand, a module typically focuses on one relatively small portion of the problem. If you're working on a single module, you'll have a smaller problem domain to wrap your head around. This makes development easier and less error-prone.

Maintainability: Modules are typically designed so that they enforce logical boundaries between different problem domains. If modules are written in a way that minimizes interdependency, there is decreased likelihood that modifications to a single module will have an impact on other parts of the program. This makes it more viable for a team of many programmers to work collaboratively on a large application.

Reusability: Functionality defined in a single module can be easily reused (through an appropriately defined interface) by other parts of the application. This eliminates the need to duplicate code.

Scoping: Modules typically define a separate namespace, which helps avoid collisions between identifiers in different areas of a program.

A Python module is a file containing Python definitions and statements. A module can define functions, classes, and variables. A module can also include runnable code. Grouping related code into a module makes the code easier to understand and use. It also makes the code logically organized.

Example: create a simple module

```
# A simple module, calc.py

def add(x, y):
    return (x+y)

def subtract(x, y):
    return (x-y)
```

Import Module in Python – Import statement
We can import the functions, classes defined in a module to another module using the **import statement** in some other Python source file.

Syntax:
import module

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches for importing a module

Example: Importing modules in Python

```
# importing  module calc.py
import calc

print(calc.add(10, 2))
```

The from import Statement
Python's from statement lets you import specific attributes from a module without importing the module as a whole.

Example: Importing specific attributes from the module

```
# importing sqrt() and factorial from the
# module math
from math import sqrt, factorial

# if we simply do "import math", then
# math.sqrt(16) and math.factorial()
# are required.
print(sqrt(16))
print(factorial(6))
```

Python - Packages

We organize a large number of files in different folders and subfolders based on some criteria, so

that we can find and manage them easily. In the same way, a package in Python takes the concept of the modular approach to next logical level. As you know, a module can contain multiple objects, such as classes, functions, etc. A package can contain one or more relevant modules. Physically, a package is actually a folder containing one or more module files.

Let's create a package named mypackage, using the following steps:

- Create a new folder named D:\MyApp.
- Inside MyApp, create a subfolder with the name 'mypackage'.
- Create an empty __init__.py file in the mypackage folder.
- Using a Python-aware editor like IDLE, create modules greet.py and functions.py with the following code:

greet.py
Copy

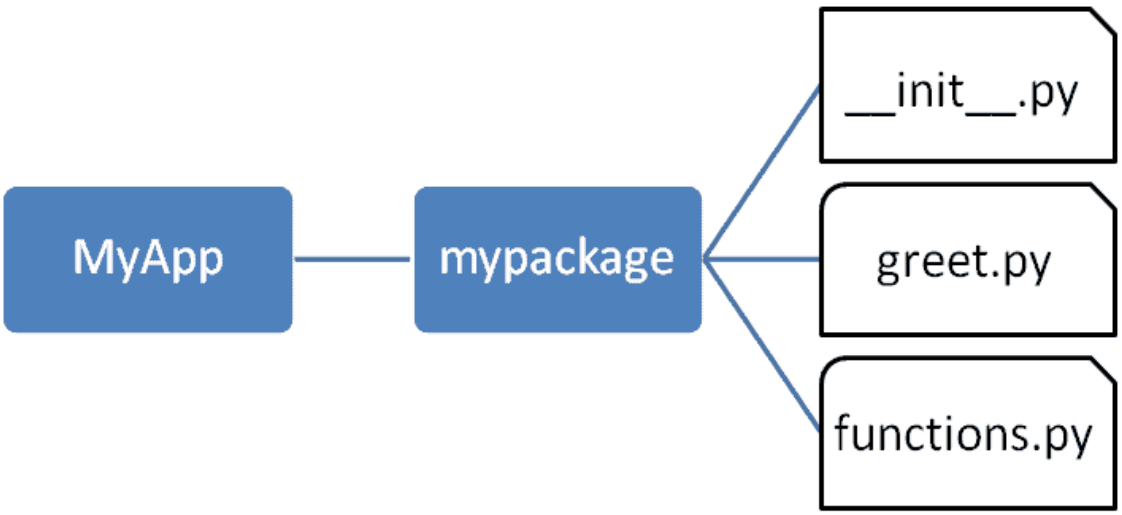
```
def SayHello(name):
   print("Hello ", name)
```

functions.py

Copy

```
def sum(x,y):
   return x+y

def average(x,y):
   return (x+y)/2

def power(x,y):
   return x**y
```

That's it. We have created our package called mypackage. The following is a folder structure:



Package Folder Structure

Importing a Module from a Package

Now, to test our package, navigate the command prompt to the MyApp folder and invoke the Python prompt from there.

D:\MyApp>python

Import the functions module from the mypackage package and call its power() function.

```
>>> from mypackage import functions
>>> functions.power(3,2)
```

init__.py

The package folder contains a special file called __init__.py, which stores the package's content. It serves two purposes:

The Python interpreter recognizes a folder as the package if it contains __init__.py file.

__init__.py exposes specified resources from its modules to be imported.

An empty __init__.py file makes all functions from the above modules available when this package is imported. Note that __init__.py is essential for the folder to be recognized by Python as a package. You can optionally define functions from individual modules to be made available.

The __init__.py file is normally kept empty. However, it can also be used to choose specific functions from modules in the package folder and make them available for import. Modify __init__.py as below:

__init__.py

Copy

```
from .functions import average, power

from .greet import SayHello
```

The specified functions can now be imported in the interpreter session or another executable script.

Create test.py in the MyApp folder to test mypackage.

Install a Package Globally

Once a package is created, it can be installed for system-wide use by running the setup script. The

script calls setup() function from the setuptools module.

Let's install mypackage for system-wide use by running a setup script.

Save the following code as setup.py in the parent folder MyApp. The script calls the setup() function from the setuptools module. The setup() function takes various arguments such as name, version, author, list of dependencies, etc. The zip_safe argument defines whether the package is installed in compressed mode or regular mode.

QUESTION : -

1. **What are modules and packages?**
2. What is a package in Python?
3. How to import all the functions in a Python module?
4. What are Python modules and packages?
5. How to use any Python source file as a module?

CONCLUSION:- ----------

Assignment No. 5

Title :- Learn Programs for implementations of all object-oriented concepts like class, method, inheritance, polymorphism etc.

.

Date:-

Marks:- Signature:-

Assignment No. 5

Problem Statement: - Programs for implementations of all object-oriented concepts like class, method, inheritance, polymorphism etc.

Objective: - Learn Programs for implementations of all object-oriented concepts like class, method, inheritance, polymorphism etc.

Theory : Object-oriented programming As the name suggests uses objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function

Like other general-purpose programming languages, Python is also an object-oriented language since its beginning. It allows us to develop applications using an Object-Oriented approach. In Python, we can easily create and use classes and objects.

An object-oriented paradigm is to design the program using classes and objects. The object is related to real-word entities such as book, house, pencil, etc. The oops concept focuses on writing the reusable code. It is a widespread technique to solve the problem by creating objects.

Major principles of object-oriented programming system are given below.

- Class
- Object
- Method
- Inheritance
- Polymorphism
- Data Abstraction
- Encapsulation

Class

The class can be defined as a collection of objects. It is a logical entity that has some specific attributes and methods. For example: if you have an employee class, then it should contain an attribute and method, i.e. an email id, name, age, salary, etc.

Syntax

```
1. class ClassName:
2.     <statement-1>
3.     .
4.     .
5.     <statement-N>
```

Object

The object is an entity that has state and behavior. It may be any real-world object like the mouse, keyboard, chair, table, pen, etc.

Everything in Python is an object, and almost everything has attributes and methods. All functions have a built-in attribute __doc__, which returns the docstring defined in the function source code.

When we define a class, it needs to create an object to allocate the memory. Consider the following example.

Example:

```
class car:
   def __init__(self,modelname, year):
      self.modelname = modelname
      self.year = year
   def display(self):
      print(self.modelname,self.year)

c1 = car("Toyota", 2016)
c1.display()
```

Inheritance

Inheritance is the most important aspect of object-oriented programming, which simulates the real-world concept of inheritance. It specifies that the child object acquires all the properties and behaviors of the parent object.

By using inheritance, we can create a class which uses all the properties and behavior of another class. The new class is known as a derived class or child class, and the one whose properties are acquired is known as a base class or parent class.

It provides the re-usability of the code.

Polymorphism

Polymorphism contains two words "poly" and "morphs". Poly means many, and morph means shape. By polymorphism, we understand that one task can be performed in different ways. For example - you have a class animal, and all animals speak. But they speak differently. Here, the "speak" behavior is polymorphic in a sense and depends on the animal. So, the abstract "animal" concept does not actually "speak", but specific animals (like dogs and cats) have a concrete implementation of the action "speak".

Encapsulation

Encapsulation is also an essential aspect of object-oriented programming. It is used to restrict access to methods and variables. In encapsulation, code and data are wrapped together within a single unit from being modified by accident.

Data Abstraction

Data abstraction and encapsulation both are often used as synonyms. Both are nearly synonyms because data abstraction is achieved through encapsulation.

Abstraction is used to hide internal details and show only functionalities. Abstracting something means to give names to things so that the name captures the core of what a function or a whole program does.

Creating classes in Python

In Python, a class can be created by using the keyword class, followed by the class name. The syntax to create a class is given below.

Syntax

```
class ClassName:
#statement_suite
```

In Python, we must notice that each class is associated with a documentation string which can be accessed by using **<class-name>.__doc__.** A class contains a statement suite including fields, constructor, function, etc. definition.

Consider the following example to create a class **Employee** which contains two fields as Employee id, and name.

The class also contains a function **display()**, which is used to display the information of the **Employee.**

Example

```
class Employee:
id = 10
name = "Devansh"
def display (self):
 print(self.id,self.name)
```

Here, the **self** is used as a reference variable, which refers to the current class object. It is always the first argument in the function definition. However, using **self** is optional in the function call.

Creating an instance of the class

A class needs to be instantiated if we want to use the class attributes in another class or method. A class can be instantiated by calling the class using the class name.

The syntax to create the instance of the class is given below.

<object-name> = <class-name>(<arguments>)

The following example creates the instance of the class Employee defined in the above example.

Example

```
class Employee:
id = 10
name = "John"
def display (self):
    print("ID: %d \nName: %s"%(self.id,self.name))
# Creating a emp instance of Employee class
emp = Employee()
emp.display()
```

QUESTION :-

1. What are Python Oops concepts?

2. What are Python Oops concepts?

3. What is the difference between object and Oops concept?

4. Is Python object-oriented?

5.What are Major principles of object-oriented programming system

CONCLUSION:- ----------

